The Poisson Distribution: A Comprehensive Examination

(Revised and corrected edition)

Fatima Emad

July 16, 2025

Abstract

The Poisson distribution models the number of events occurring in a fixed interval when events arrive independently at a constant average rate. This paper gives a self-contained examination of its fundamental properties, with complete mathematical proofs, numerical illustrations, and its relationship to the binomial, exponential and normal distributions. We distinguish carefully between the reliability and hazard functions of a Poisson *process* (waiting times) and the discrete hazard of the *count*. We then present Poisson regression and five applications—call-centre load, security-breach detection, traffic flow, disease surveillance and e-commerce clickstreams—implemented in Python on synthetic data with known ground truth, including a diagnostic for overdispersion. All code and figures are reproducible from the accompanying notebook.

1 Introduction

The Poisson distribution describes the count of events in a specified interval of time or space, under the assumptions that events occur independently and at a constant average rate. It is named after Siméon Denis Poisson and is used throughout biology, telecommunications, operations research and machine learning [1, 2].

1.1 Significance

Many real-world measurements are *counts*: calls received, vehicles passing, failures observed, cases reported. Counts are non-negative integers, so the normal distribution is a poor model for small counts. The Poisson law is the basic model for such data and the foundation of Poisson regression, a generalised linear model (GLM) [4, 5]; see also [7, 8, 9].

1.2 About "rare events"

The Poisson law is often described as the law of *rare* events. This comes from its derivation as the limit of $\mathrm{Bin}(n, p)$ with n large and p small (Section 4). Rarity is not required, however: the law is valid for any $\lambda > 0$ provided the assumptions of Section 2 hold. For example, hourly call counts with $\lambda \approx 20$ are perfectly well modelled by it.

2 Key characteristics

1. **Independence.** The occurrence of one event does not change the probability of another.
2. **Constant average rate λ.** The expected number of events per interval is the same for all intervals of equal length (a *homogeneous* Poisson process). When the rate varies over time, one uses Poisson regression or a non-homogeneous process (Section 9.1).

3. **Events occur one at a time.** Simultaneous events have negligible probability.

4. **Equidispersion.** Mean and variance are equal, $E[X] = \mathrm{Var}(X) = \lambda$. This is the assumption most often violated by real data (Section 7.5).

3 Mathematical properties

Let $X \sim \mathrm{Poisson}(\lambda)$. Table 1 summarises the properties proved in Section 4.

Property	Formula
PMF	$P(X = k) = \dfrac{e^{-\lambda}\lambda^k}{k!}, \quad k = 0, 1, 2, \ldots$
CDF	$F(k) = P(X \le k) = \displaystyle\sum_{i=0}^{k} \frac{e^{-\lambda}\lambda^i}{i!} = \frac{\Gamma(k+1, \lambda)}{k!}$
MGF	$M(t) = \exp\big(\lambda(e^t - 1)\big)$
Mean, variance, std. dev.	$E[X] = \lambda, \quad \mathrm{Var}(X) = \lambda, \quad \sigma = \sqrt{\lambda}$
Mode	$\lfloor \lambda \rfloor$ if $\lambda \notin \mathbb{Z}$; $\lambda - 1$ and λ if $\lambda \in \mathbb{Z}$
Waiting time (A)	$R(t) = P(T > t) = e^{-\lambda t}, \quad Z(t) = \lambda$
Discrete count (B)	$S(k) = P(X > k), \quad h(k) = \dfrac{P(X = k)}{P(X \ge k)}$

Table 1: Summary of properties. Rows (A) and (B) are two distinct notions of "reliability" and "hazard" (Section 4.8).

Reliability and hazard: two settings. In a Poisson *process* with rate λ per unit time, the time T to the first event satisfies $\{T > t\} = \{N(t) = 0\}$, hence $R(t) = e^{-\lambda t}$ and the hazard $Z(t) = f(t)/R(t) = \lambda$ is constant. If instead the *count* k is treated as a discrete lifetime, the survival function is $S(k) = P(X > k)$ and the hazard is the conditional probability $h(k) = P(X = k \mid X \ge k)$, which lies in $(0, 1]$ and increases with k. The first version of this paper used $P(X = t)/P(X > t)$ for the hazard and called t both a count and a time; that ratio is not a probability and can exceed 1.

4 Proofs

Throughout, $X \sim \mathrm{Poisson}(\lambda)$ with $\lambda > 0$, and $p_k = P(X = k) = e^{-\lambda}\lambda^k/k!$ for $k = 0, 1, 2, \ldots$ We use the exponential series $e^x = \sum_{j \ge 0} x^j/j!$, valid for every real x.

4.1 Probability mass function (Poisson limit theorem)

Theorem 1. *Let $X_n \sim \mathrm{Bin}(n, \lambda/n)$. For every fixed $k \in \{0, 1, 2, \ldots\}$,*

$$\lim_{n \to \infty} P(X_n = k) = \frac{e^{-\lambda}\lambda^k}{k!}.$$

Proof.

$$P(X_n = k) = \binom{n}{k}\left(\frac{\lambda}{n}\right)^k\left(1-\frac{\lambda}{n}\right)^{n-k}$$
$$= \frac{\lambda^k}{k!}\cdot\underbrace{\frac{n(n-1)\cdots(n-k+1)}{n^k}}_{=\prod_{j=0}^{k-1}(1-j/n)}\cdot\left(1-\frac{\lambda}{n}\right)^n\cdot\left(1-\frac{\lambda}{n}\right)^{-k}.$$

As $n\to\infty$ with k fixed: the product of k factors tends to 1; $(1-\lambda/n)^n\to e^{-\lambda}$; and $(1-\lambda/n)^{-k}\to 1$. Hence $P(X_n=k)\to e^{-\lambda}\lambda^k/k!$. □

Remark 1. (i) The p_k form a probability distribution because $\sum_{k\geq 0}p_k = e^{-\lambda}\sum_{k\geq 0}\lambda^k/k! = e^{-\lambda}e^{\lambda} = 1$. (ii) *Poisson process.* Suppose events in $[0,t]$ occur independently, at most one per small sub-interval, with probability $\lambda t/n$ in each of n equal sub-intervals. Then the count $N(t)$ is $\mathrm{Bin}(n,\lambda t/n)$, and letting $n\to\infty$ gives $N(t)\sim\mathrm{Poisson}(\lambda t)$, i.e. $P(N(t)=k)=e^{-\lambda t}(\lambda t)^k/k!$.

4.2 Cumulative distribution function

By definition

$$F(k) = P(X\leq k) = \sum_{i=0}^{k}\frac{e^{-\lambda}\lambda^i}{i!}, \qquad P(X>k) = 1-F(k) = \sum_{i=k+1}^{\infty}\frac{e^{-\lambda}\lambda^i}{i!}.$$

(The exponential series shows that the two sums are complementary, since the full series sums to 1.)

Proposition 2 (Incomplete-gamma form)**.** $F(k) = \dfrac{1}{k!}\displaystyle\int_\lambda^\infty s^k e^{-s}\,ds = \frac{\Gamma(k+1,\lambda)}{k!}$. *In particular F is strictly decreasing in λ.*

Proof. Differentiate in λ; each term gives $\frac{d}{d\lambda}\frac{e^{-\lambda}\lambda^i}{i!} = -\frac{e^{-\lambda}\lambda^i}{i!}+\frac{e^{-\lambda}\lambda^{i-1}}{(i-1)!}$ (the last term is absent for $i=0$). Summing over $i=0,\ldots,k$ the sum telescopes: $\frac{d}{d\lambda}F(k) = -\frac{e^{-\lambda}\lambda^k}{k!}<0$. Since $F(k)\to 0$ as $\lambda\to\infty$, integrating from λ to ∞ gives $F(k)=\int_\lambda^\infty\frac{e^{-s}s^k}{k!}ds$. □

4.3 Moment generating function

Theorem 3. $M(t) = E[e^{tX}] = \exp\big(\lambda(e^t-1)\big)$ *for all real t.*

Proof.

$$M(t) = \sum_{k=0}^{\infty}e^{tk}\frac{e^{-\lambda}\lambda^k}{k!} = e^{-\lambda}\sum_{k=0}^{\infty}\frac{(\lambda e^t)^k}{k!} = e^{-\lambda}e^{\lambda e^t} = e^{\lambda(e^t-1)}.$$ □

4.4 Mean

Theorem 4. $E[X]=\lambda$.

Proof. The $k=0$ term vanishes, and $k/k! = 1/(k-1)!$ for $k\geq 1$. Substituting $j=k-1$:

$$E[X] = \sum_{k=1}^{\infty}k\,\frac{e^{-\lambda}\lambda^k}{k!} = e^{-\lambda}\sum_{k=1}^{\infty}\frac{\lambda^k}{(k-1)!} = \lambda e^{-\lambda}\sum_{j=0}^{\infty}\frac{\lambda^j}{j!} = \lambda e^{-\lambda}e^{\lambda} = \lambda.$$ □

4.5 Variance

Theorem 5. $\mathrm{Var}(X) = \lambda$.

Proof. Write $k^2 = k(k-1) + k$. For $k \geq 2$, $k(k-1)/k! = 1/(k-2)!$, so with $j = k-2$

$$E[X(X-1)] = e^{-\lambda} \sum_{k=2}^{\infty} \frac{\lambda^k}{(k-2)!} = \lambda^2 e^{-\lambda} \sum_{j=0}^{\infty} \frac{\lambda^j}{j!} = \lambda^2.$$

Hence $E[X^2] = E[X(X-1)] + E[X] = \lambda^2 + \lambda$ and $\mathrm{Var}(X) = E[X^2] - (E[X])^2 = \lambda^2 + \lambda - \lambda^2 = \lambda$.

Cross-check via the MGF: $M'(0) = \lambda$ and $M''(0) = \lambda^2 + \lambda$ (differentiate $M(t) = e^{\lambda(e^t - 1)}$ twice), giving the same values. □

4.6 Standard deviation

$\sigma = \sqrt{\mathrm{Var}(X)} = \sqrt{\lambda}$. The equality of mean and variance (*equidispersion*) is the property tested by the dispersion diagnostic used in the regression applications of the paper.

4.7 Mode

Theorem 6. *If $\lambda \notin \mathbb{Z}$, the unique mode is $\lfloor \lambda \rfloor$. If $\lambda \in \mathbb{Z}$, the modes are $\lambda - 1$ and λ.*

Proof. For $k \geq 1$ the ratio of consecutive probabilities is

$$\frac{p_k}{p_{k-1}} = \frac{e^{-\lambda}\lambda^k / k!}{e^{-\lambda}\lambda^{k-1}/(k-1)!} = \frac{\lambda}{k}.$$

So $p_k > p_{k-1}$ iff $k < \lambda$, $p_k = p_{k-1}$ iff $k = \lambda$, and $p_k < p_{k-1}$ iff $k > \lambda$. The sequence therefore increases while $k < \lambda$ and decreases once $k > \lambda$. If λ is not an integer, the last increase is at $k = \lfloor \lambda \rfloor$ and the next step is a decrease, so $\lfloor \lambda \rfloor$ is the unique maximum. If λ is an integer, $p_\lambda = p_{\lambda - 1}$ and both are maximal (for $\lambda < 1$ the sequence decreases from $k = 0$, so the mode is $0 = \lfloor \lambda \rfloor$). □

4.8 Reliability and hazard functions

Two different objects are called "reliability" in connection with the Poisson law; they must not be mixed.

(A) Waiting time in a Poisson process. Let events occur at rate λ per unit time and let T be the time of the first event. Then $\{T > t\} = \{N(t) = 0\}$, so by the Remark in Section 4.1

$$R(t) = P(T > t) = e^{-\lambda t}, \qquad f(t) = -R'(t) = \lambda e^{-\lambda t}, \qquad Z(t) = \frac{f(t)}{R(t)} = \lambda.$$

So $T \sim \mathrm{Exp}(\lambda)$ and the hazard is the constant λ (the memoryless property). This is the reliability function used in reliability engineering.

(B) The count as a discrete "lifetime". Using the index k itself as the variable, define the survival function and the *discrete hazard*

$$S(k) = P(X > k) = 1 - F(k),$$
$$h(k) = P(X = k \mid X \geq k) = \frac{P(X = k)}{P(X \geq k)} = \frac{p_k}{1 - F(k-1)}, \qquad F(-1) := 0.$$

Note that the denominator is $P(X \geq k)$, not $P(X > k)$: a hazard is a *conditional probability*, so it lies in $(0, 1]$. (The ratio $p_k/P(X > k)$ is not a probability and can exceed 1; for $\lambda = 5$ it reaches about 3.06 at $k = 19$.)

Proposition 7. *$h(0) = e^{-\lambda}$; $0 < h(k) < 1$ for every k; h is strictly increasing in k; and $h(k) \to 1$ as $k \to \infty$.*

Proof. $h(0) = p_0/P(X \geq 0) = e^{-\lambda}$. Since $P(X \geq k) = p_k + P(X > k)$ and $P(X > k) > 0$, we get $h(k) < 1$. Next,

$$\frac{1}{h(k)} = \sum_{j=0}^{\infty} \frac{p_{k+j}}{p_k} = \sum_{j=0}^{\infty} \frac{\lambda^j \, k!}{(k+j)!} = 1 + \sum_{j=1}^{\infty} \frac{\lambda^j}{(k+1)(k+2)\cdots(k+j)}.$$

For each $j \geq 1$ the term decreases strictly in k, so $1/h(k)$ strictly decreases, i.e. h strictly increases. Each term is also bounded by $\lambda^j/j!$ (summable) and tends to 0 as $k \to \infty$, so by dominated convergence $1/h(k) \to 1$. □

Remark 2. The *increasing* hazard in (B) is a statement about how the conditional chance of "stopping at k" grows with k; it is a property of the count distribution, not of waiting time. Contrast it with the constant hazard λ in (A).

4.9 Normal approximation for large λ

Theorem 8. *If $Z_\lambda = (X - \lambda)/\sqrt{\lambda}$, then $Z_\lambda \Rightarrow N(0, 1)$ as $\lambda \to \infty$. Hence $X \approx N(\mu = \lambda,\ \sigma^2 = \lambda)$.*

Proof. With $u = s/\sqrt{\lambda}$, the MGF of Z_λ is

$$M_{Z_\lambda}(s) = e^{-s\sqrt{\lambda}} \, e^{\lambda(e^u - 1)} = \exp\big(\lambda(e^u - 1 - u)\big),$$

and $e^u - 1 - u = u^2/2 + O(u^3)$, so $\lambda(e^u - 1 - u) = s^2/2 + O(\lambda^{-1/2}) \to s^2/2$. Thus $M_{Z_\lambda}(s) \to e^{s^2/2}$, the MGF of $N(0, 1)$, and the claim follows from Lévy's continuity theorem. □

In practice a continuity correction improves the approximation: $P(X \leq k) \approx \Phi\big((k + \tfrac{1}{2} - \lambda)/\sqrt{\lambda}\big)$.

5 Numerical illustration

All figures in this paper are produced by the companion notebook (single seeded generator, so results are reproducible).

5.1 PMF and CDF

```
lam = 5
n = 1000
sample = rng.poisson(lam, n)

k = np.arange(0, 20)
emp_pmf = np.bincount(sample, minlength=k.size)[:k.size] / n

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))

# Left: observed frequencies vs. theoretical PMF
ax[0].bar(k, emp_pmf, width=0.8, alpha=0.6, label="Simulated frequencies (n=1000)")
ax[0].plot(k, poisson.pmf(k, lam), "ro-", label=r"Poisson PMF ($\lambda=5$)")
```

```
ax[0].set_xlabel("Event count k")
ax[0].set_ylabel("Probability")
ax[0].set_title("PMF: simulation vs. theory")
ax[0].legend()

# Right: CDF and the smallest k reaching 95%
k95 = int(poisson.ppf(0.95, lam))
ax[1].step(k, poisson.cdf(k, lam), where="post", color="purple", label="CDF")
ax[1].axhline(0.95, color="red", ls="--", label="95% level")
ax[1].axvline(k95, color="gray", ls=":", label=f"smallest k with CDF >= 95%: {k95}")
ax[1].set_xlabel("Event count k")
ax[1].set_ylabel("Cumulative probability")
ax[1].set_title("CDF")
ax[1].legend(loc="lower right")

plt.tight_layout()
plt.show()

print("sample mean :", sample.mean(), " (theory:", lam, ")")
print("sample variance:", round(sample.var(ddof=1), 3), " (theory:", lam, ")")
print("P(X <= k95) :", round(poisson.cdf(k95, lam), 4))
print("CDF = incomplete-gamma identity holds:",
      np.allclose(poisson.cdf(k, lam), gammaincc(k + 1, lam)))
```

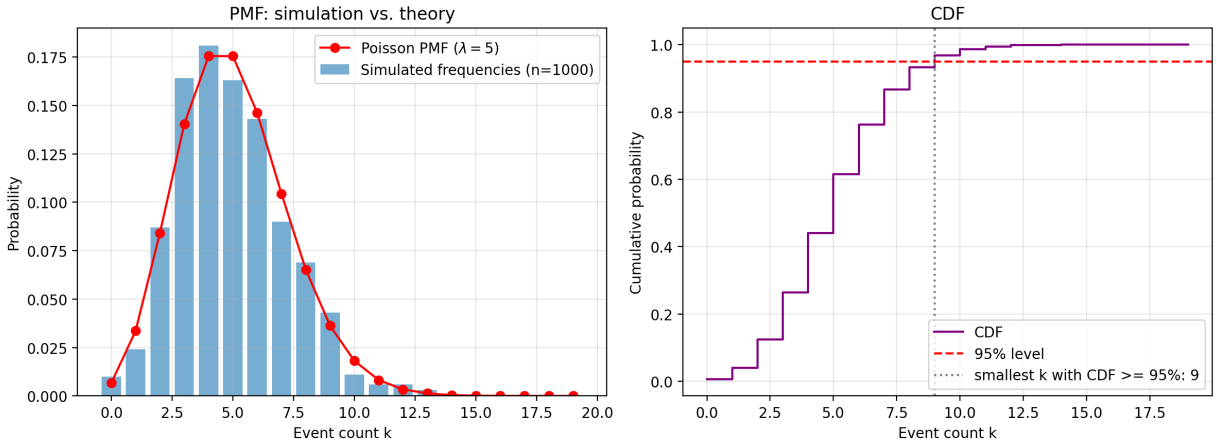


Figure 1: Left: relative frequencies of 1000 simulated Poisson(5) draws against the theoretical PMF (bars are aligned to the integers). Right: the CDF; the smallest k with $F(k) \geq 0.95$ is $k = 9$.

The sample mean is 4.943 and the sample variance 5.045, both close to $\lambda = 5$. The CDF at $k = 9$ equals 0.9682, and the code confirms numerically the incomplete-gamma identity $F(k) = \Gamma(k+1, \lambda)/k!$.

5.2 Survival and hazard of the count

```
k = np.arange(0, 20)
surv = poisson.sf(k, lam) # S(k) = P(X > k)
hazard = poisson.pmf(k, lam) / poisson.sf(k - 1, lam) # h(k) = P(X=k) / P(X>=k)
wrong = poisson.pmf(k, lam) / poisson.sf(k, lam) # P(X=k) / P(X>k) -> NOT a probability

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))

ax[0].plot(k, surv, "bo-", label=r"$S(k)=P(X>k)$")
ax[0].axhline(0.05, color="red", ls="--", label="5% level")
ax[0].set_xlabel("k")
ax[0].set_ylabel("S(k)")
```

```
ax[0].set_title(r"Survival function, $\lambda=5$")
ax[0].legend()

ax[1].plot(k, hazard, "go-", label=r"$h(k)=P(X=k)/P(X\geq k)$ (correct)")
ax[1].plot(k, wrong, color="gray", ls="--", marker="x", label=r"$P(X=k)/P(X>k)$ (invalid)")
ax[1].axhline(1, color="black", ls=":", lw=1)
ax[1].set_ylim(0, 3.3)
ax[1].set_xlabel("k")
ax[1].set_ylabel("hazard")
ax[1].set_title("Discrete hazard")
ax[1].legend(loc="upper left")

plt.tight_layout()
plt.show()

print("h(0) = e^-lambda =", round(hazard[0], 5), "=", round(np.exp(-lam), 5))
print("max of correct hazard:", round(hazard.max(), 4), "| max of invalid ratio:", round(wrong.
    max(), 2))
print("hazard is increasing:", bool(np.all(np.diff(hazard) > 0)))
```

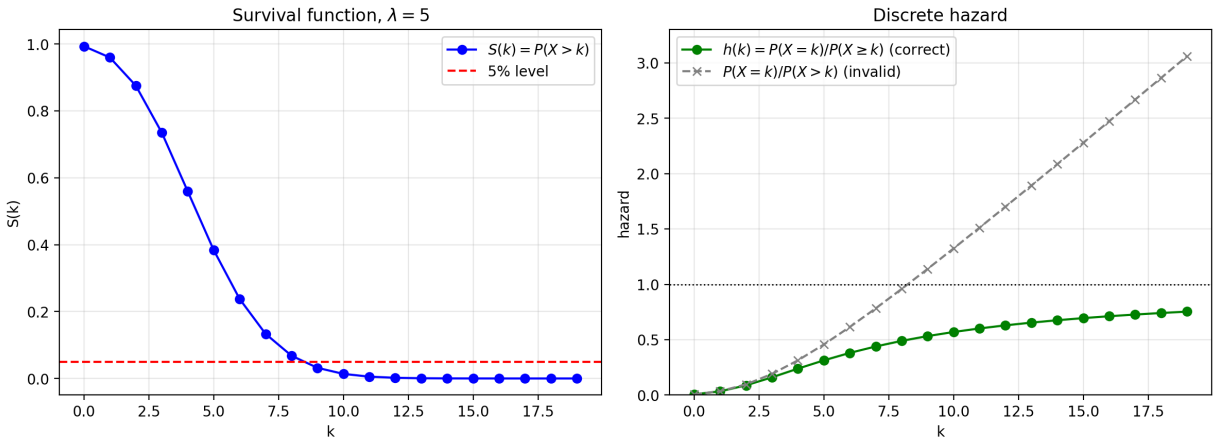


Figure 2: Left: survival function $S(k) = P(X > k)$ for $\lambda = 5$. Right: the correct discrete hazard $h(k) = P(X = k)/P(X \geq k)$ (green) stays below 1 and increases; the invalid ratio $P(X = k)/P(X > k)$ (grey, dashed) exceeds 1 and reaches 3.06 at $k = 19$.

The hazard starts at $h(0) = e^{-5} = 0.00674$, is increasing, and has maximum 0.7537 on the plotted range, in agreement with the proposition in Section 4.8.

5.3 Reliability and hazard of the waiting time

```
rate = 5.0 # events per unit time
t = np.linspace(0, 1.5, 200)

T = rng.exponential(1 / rate, 5000) # simulated waiting times to the first event
R_emp = np.array([(T > ti).mean() for ti in t])
R_th = np.exp(-rate * t)

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))

ax[0].plot(t, R_th, "b-", label=r"theory $R(t)=e^{-\lambda t}$")
ax[0].plot(t, R_emp, "k.", ms=3, label="simulation (5000 runs)")
ax[0].set_xlabel("time t")
ax[0].set_ylabel("R(t)")
ax[0].set_title("Reliability of a Poisson process")
ax[0].legend()
```

```
ax[1].plot(t, np.full_like(t, rate), "g-", label=r"$Z(t)=\lambda$")
ax[1].set_ylim(0, 8)
ax[1].set_xlabel("time t")
ax[1].set_ylabel("Z(t)")
ax[1].set_title("Constant hazard (memoryless)")
ax[1].legend()

plt.tight_layout()
plt.show()

print("mean waiting time: simulated", round(T.mean(), 4), "| theory 1/lambda =", 1 / rate)
```

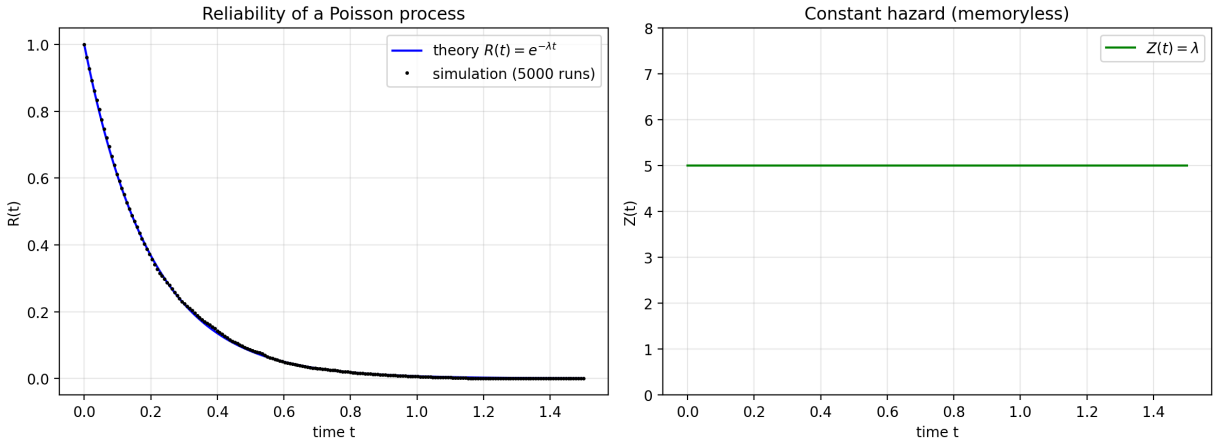


Figure 3: Left: simulated survival of the waiting time to the first event (5000 runs) against $R(t) = e^{-\lambda t}$. Right: the constant hazard $Z(t) = \lambda$.

The simulated mean waiting time is 0.2008, against the theoretical $1/\lambda = 0.2$.

6 Relationships with other distributions

Binomial. If n is large, p small and $np = \lambda$ is fixed, $\mathrm{Bin}(n, p)$ is approximated by $\mathrm{Poisson}(\lambda)$ (proved in Section 4.1).

Exponential. The waiting times between events of a Poisson process are i.i.d. $\mathrm{Exp}(\lambda)$ with density $f(t) = \lambda e^{-\lambda t}$, $t \geq 0$ (Section 4.8). This links the Poisson law to queueing theory and reliability engineering [3].

Normal. For large λ, $X \approx N(\mu = \lambda, \sigma^2 = \lambda)$ (proved above); with the continuity correction $P(X \leq k) \approx \Phi\big((k + \frac{1}{2} - \lambda)/\sqrt{\lambda}\big)$.

```
# (a) Binomial(n=1000, p=0.005) vs Poisson(5)
n_trials, p = 1000, 0.005
kb = np.arange(0, 16)
tv = 0.5 * np.abs(binom.pmf(kb, n_trials, p) - poisson.pmf(kb, n_trials * p)).sum()

# (b) Poisson(30) vs Normal(30, 30)
lam_b = 30
kn = np.arange(0, 70)

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
```

```
ax[0].bar(kb - 0.2, binom.pmf(kb, n_trials, p), width=0.4, label="Binomial(1000, 0.005)")
ax[0].bar(kb + 0.2, poisson.pmf(kb, n_trials * p), width=0.4, label=r"Poisson($\lambda=5$)")
ax[0].set_xlabel("k")
ax[0].set_ylabel("P(X = k)")
ax[0].set_title("Binomial vs. Poisson")
ax[0].legend()

ax[1].bar(kn, poisson.pmf(kn, lam_b), alpha=0.6, label=r"Poisson($\lambda=30$)")
xs = np.linspace(0, 70, 400)
ax[1].plot(xs, norm.pdf(xs, lam_b, np.sqrt(lam_b)), "r-", label=r"Normal($\mu=30,\ \sigma^2=30$)
    ")
ax[1].set_xlabel("k")
ax[1].set_ylabel("probability / density")
ax[1].set_title("Poisson vs. Normal")
ax[1].legend()

plt.tight_layout()
plt.show()

print("total variation distance, Bin(1000,0.005) vs Poisson(5):", round(tv, 5))
for lam_ in (5, 30, 100):
    ks = np.arange(0, int(lam_ + 8 * np.sqrt(lam_)))
    exact = poisson.cdf(ks, lam_)
    approx = norm.cdf((ks + 0.5 - lam_) / np.sqrt(lam_))
    print(f"lambda={lam_:>3}: max CDF error of the normal approximation = {np.abs(exact - approx)
        .max():.4f}")
```

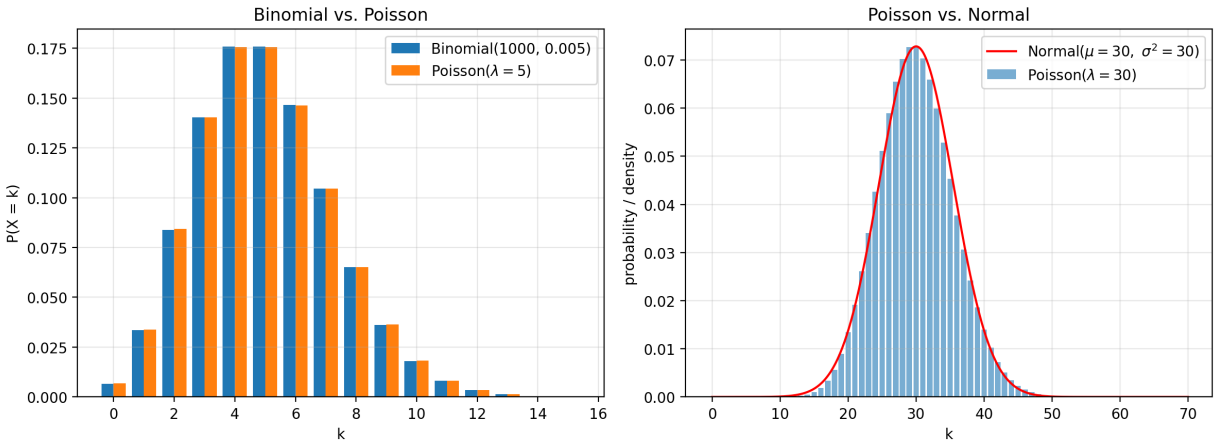


Figure 4: Left: $\mathrm{Bin}(1000, 0.005)$ and $\mathrm{Poisson}(5)$ are almost indistinguishable (total variation distance 0.0012). Right: $\mathrm{Poisson}(30)$ with the normal density of the same mean and variance.

The largest error of the continuity-corrected normal approximation of the CDF is 0.0290 for $\lambda = 5$, 0.0121 for $\lambda = 30$ and 0.0066 for $\lambda = 100$: it improves as λ grows.

7 Poisson regression and applications

7.1 The model

For counts y_i with covariates x_i, Poisson regression assumes

$$y_i \sim \mathrm{Poisson}(\mu_i), \qquad \log \mu_i = x_i^\top \beta.$$

The coefficients are estimated by maximum likelihood (iteratively re-weighted least squares) using the canonical log link [4, 6]. Because of the log link, e^{β_j} is the *multiplicative* change in the expected

count for a one-unit increase in x_j. Cyclic patterns (hour of day, week of year) are captured with Fourier terms $\sin(2\pi hx/P)$, $\cos(2\pi hx/P)$. When intervals have different lengths or populations, an offset log(exposure) is added to the linear predictor.

Dispersion diagnostic. The Pearson χ^2 statistic divided by the residual degrees of freedom is about 1 if the Poisson assumption holds; values well above 1 indicate overdispersion, in which case the Poisson standard errors are too small.

About the data. All datasets are *synthetic*, generated with known true coefficients so that parameter recovery can be checked. The first version generated data with a constant rate, so the fitted regression line was flat by construction; here every application contains a genuine signal.

```
def fourier_design(x, period, harmonics=1):
    # Intercept + sin/cos terms: lets a log-linear model learn a cyclic pattern.
    cols = [np.ones(len(x))]
    for h in range(1, harmonics + 1):
        cols += [np.sin(2 * np.pi * h * x / period), np.cos(2 * np.pi * h * x / period)]
    return np.column_stack(cols)

def dispersion(result):
    # Pearson chi-square / residual d.o.f. (about 1 if the Poisson assumption holds)
    return result.pearson_chi2 / result.df_resid
```

7.2 Customer-support call volume

Use case: staffing. Calls per hour follow a daily cycle; one harmonic captures it.

```
hour = np.tile(np.arange(24), 100) # 100 days of hourly counts
X = fourier_design(hour, period=24, harmonics=1)
beta_true = np.array([1.5, 0.5, -0.5]) # true log-rate coefficients
calls = rng.poisson(np.exp(X @ beta_true))

model = sm.GLM(calls, X, family=sm.families.Poisson()).fit()
print("true beta :", beta_true)
print("estimated beta:", np.round(model.params, 3))
print("dispersion :", round(dispersion(model), 3))

hours = np.arange(24)
Xh = fourier_design(hours, 24, 1)
fitted = np.exp(Xh @ model.params)
true_rate = np.exp(Xh @ beta_true)
observed_mean = calls.reshape(100, 24).mean(axis=0)

plt.figure(figsize=(10, 5))
plt.scatter(hour + rng.uniform(-0.25, 0.25, hour.size), calls, s=8, alpha=0.25, label="Observed
    calls (jittered)")
plt.plot(hours, observed_mean, "ko", label="Hourly mean of observations")
plt.plot(hours, fitted, "r-", lw=2, label="Poisson regression fit")
plt.plot(hours, true_rate, "g--", lw=2, label="True rate")
plt.title("Customer support call volume")
plt.xlabel("Hour of day")
plt.ylabel("Calls per hour")
plt.legend()
plt.show()

print("busiest hour (fit):", int(hours[fitted.argmax()]), "| expected calls:", round(fitted.max
    (), 2))
```

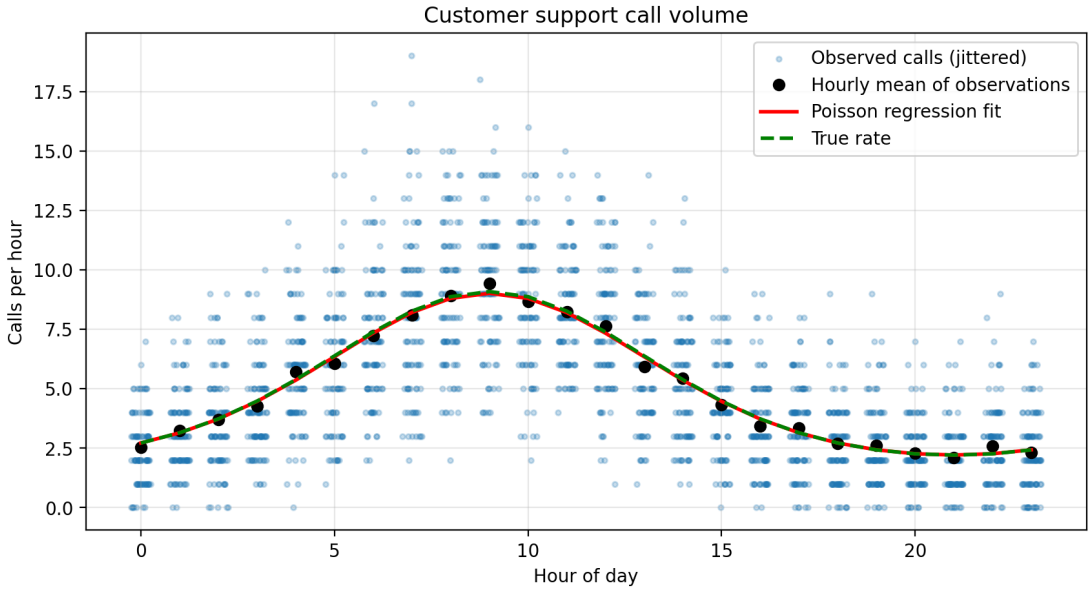


Figure 5: Observed hourly calls over 100 days (jittered), hourly means, the fitted Poisson regression and the true rate.

The estimated coefficients $(1.497, 0.497, -0.496)$ are very close to the true $(1.5, 0.5, -0.5)$, and the dispersion is 1.033. The busiest hour is 09:00 with about 9.0 expected calls, which can be used to schedule staff.

7.3 Security breach detection with Poisson tail probabilities

Use case: flag time windows whose number of breaches is implausibly high. We estimate $\hat{\lambda}$ on a clean training period and flag a window with count k when $P(X \geq k \mid \hat{\lambda}) < \alpha$. Unlike an Isolation Forest with a fixed `contamination`, the false-alarm rate is governed by α and the method does not label a fixed percentage of the data as anomalous (the first version's `contamination=0.05` did).

```
n_windows = 1000
counts = rng.poisson(0.5, n_windows) # normal behaviour: 0.5 breaches / window
attack_idx = rng.choice(np.arange(500, n_windows), 8, replace=False)
counts[attack_idx] = rng.poisson(6, 8) # injected attack windows
is_attack = np.zeros(n_windows, dtype=bool)
is_attack[attack_idx] = True

lam_hat = counts[:500].mean() # clean training period
alpha = 1e-3
flag = poisson.sf(counts - 1, lam_hat) < alpha # P(X >= k) < alpha
k_crit = int(np.argmax(poisson.sf(np.arange(50) - 1, lam_hat) < alpha))

tp = int((flag & is_attack).sum())
fp = int((flag & ~is_attack).sum())
fn = int((~flag & is_attack).sum())
print("lambda_hat =", round(lam_hat, 3), "| alert when count >=", k_crit)
print(f"true attacks: {is_attack.sum()} | detected: {tp} | false alarms: {fp} | missed: {fn}")

plt.figure(figsize=(12, 5))
plt.scatter(np.where(~flag)[0], counts[~flag], s=10, color="tab:blue", label="Normal")
plt.scatter(np.where(flag)[0], counts[flag], s=50, color="tab:red", label="Flagged as anomaly")
plt.axhline(k_crit - 0.5, color="red", ls="--", label=f"Alert level (count >= {k_crit}, alpha =
    {alpha})")
plt.title("Network security breach detection")
```

```
plt.xlabel("Time window")
plt.ylabel("Breaches per window")
plt.legend()
plt.show()
```

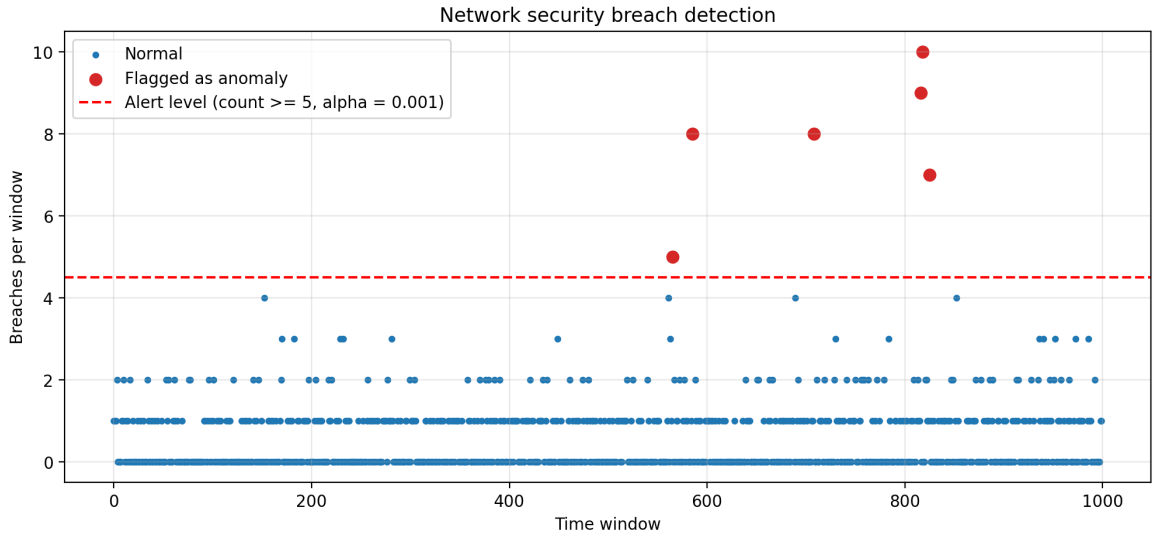


Figure 6: Breach counts per window; red points exceed the alert level (count ≥ 5 for $\alpha = 0.001$).

With $\hat{\lambda} = 0.492$ the alert level is 5 breaches per window. Of the 8 injected attack windows, 6 were detected, there were 0 false alarms and 2 attacks were missed (attack windows with small counts are indistinguishable from normal variation at this α). The trade-off between detection and false alarms is controlled explicitly through α.

7.4 Traffic flow

Use case: vehicle arrivals at an intersection have morning and evening peaks, so two harmonics are used.

```
time_of_day = np.tile(np.arange(24), 100)
X = fourier_design(time_of_day, period=24, harmonics=2)
beta_true = np.array([2.6, -0.06, -0.24, -0.2, -0.35]) # peaks near 08:00 and 18:00
vehicles = rng.poisson(np.exp(X @ beta_true))

traffic_model = sm.GLM(vehicles, X, family=sm.families.Poisson()).fit()
print("true beta :", beta_true)
print("estimated beta:", np.round(traffic_model.params, 3))
print("dispersion :", round(dispersion(traffic_model), 3))

hours = np.arange(24)
Xh = fourier_design(hours, 24, 2)
fitted = np.exp(Xh @ traffic_model.params)
observed_mean = vehicles.reshape(100, 24).mean(axis=0)

plt.figure(figsize=(10, 5))
plt.scatter(time_of_day + rng.uniform(-0.25, 0.25, time_of_day.size), vehicles, s=8, alpha=0.25,
            label="Observed vehicles (jittered)")
plt.plot(hours, observed_mean, "ko", label="Hourly mean of observations")
plt.plot(hours, fitted, "g-", lw=2, label="Poisson regression fit (2 harmonics)")
plt.title("Traffic flow analysis")
plt.xlabel("Hour of day")
plt.ylabel("Vehicles per interval")
plt.legend()
```

```
plt.show()

print("busiest hour (fit):", int(hours[fitted.argmax()]), "| expected vehicles:", round(fitted.
    max(), 1))
```

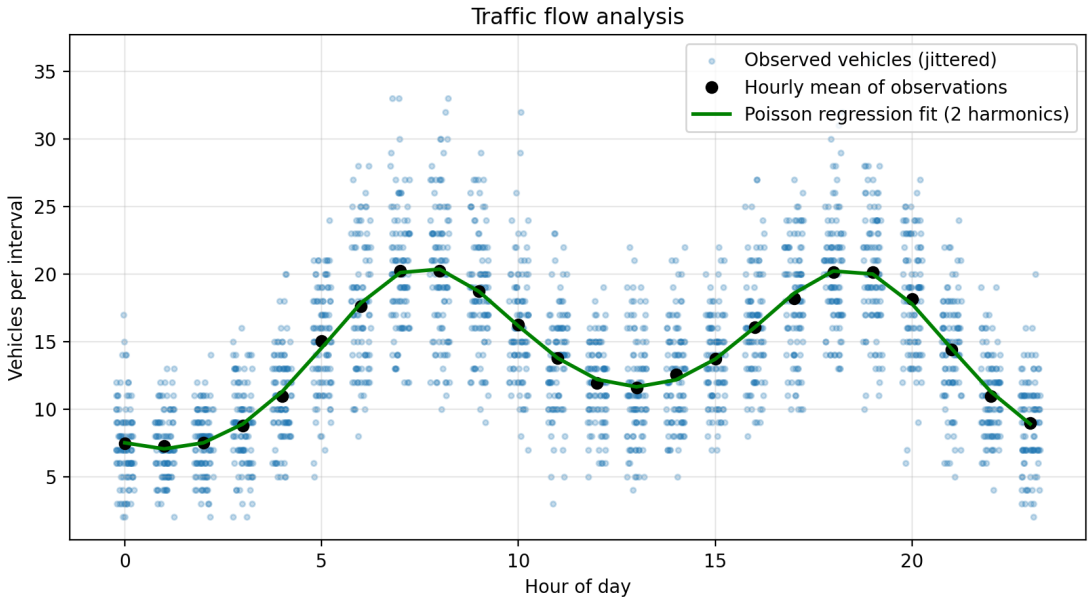


Figure 7: Observed vehicle counts, hourly means and the two-harmonic Poisson regression fit.

The model recovers the double-peaked profile (peaks at about 08:00 and 18:00, with about 20 vehicles per interval at the maximum), the coefficients match the truth to within 0.01, and the dispersion is 1.001.

7.5 Disease incidence and overdispersion

Use case: weekly case counts with a seasonal pattern. Surveillance counts are typically *overdispersed* because cases cluster. We simulate this with a Negative Binomial, fit a Poisson model, and show the consequences; a quasi-Poisson correction (`scale="X2"`) repairs the standard errors.

```
weeks = np.arange(1, 157) # three years of weekly counts
X = fourier_design(weeks, period=52, harmonics=1)
beta_true = np.array([1.2, 0.7, 0.4])
mu = np.exp(X @ beta_true)
r = 1.5 # NB size: Var = mu + mu^2 / r (overdispersed)
cases = rng.negative_binomial(r, r / (r + mu))

pois = sm.GLM(cases, X, family=sm.families.Poisson()).fit()
quasi = sm.GLM(cases, X, family=sm.families.Poisson()).fit(scale="X2")

print("true beta :", beta_true)
print("estimated beta :", np.round(pois.params, 3))
print("dispersion :", round(dispersion(pois), 2), "(1 = Poisson-consistent)")
print("std errors Poisson :", np.round(pois.bse, 3))
print("std errors quasi-Poisson:", np.round(quasi.bse, 3))

fitted = pois.fittedvalues
lo, hi = poisson.ppf(0.025, fitted), poisson.ppf(0.975, fitted)
inside = np.mean((cases >= lo) & (cases <= hi))
print(f"share of weeks inside the Poisson 95% band: {inside:.2f} (expected about 0.95)")
```

```
plt.figure(figsize=(12, 5))
plt.fill_between(weeks, lo, hi, color="orange", alpha=0.25, label="Poisson 95% band")
plt.plot(weeks, cases, "o", ms=4, alpha=0.7, label="Observed weekly cases")
plt.plot(weeks, fitted, color="darkorange", lw=2, label="Poisson regression fit")
plt.title("Disease incidence tracking (overdispersed counts)")
plt.xlabel("Week")
plt.ylabel("Cases per week")
plt.legend()
plt.show()
```

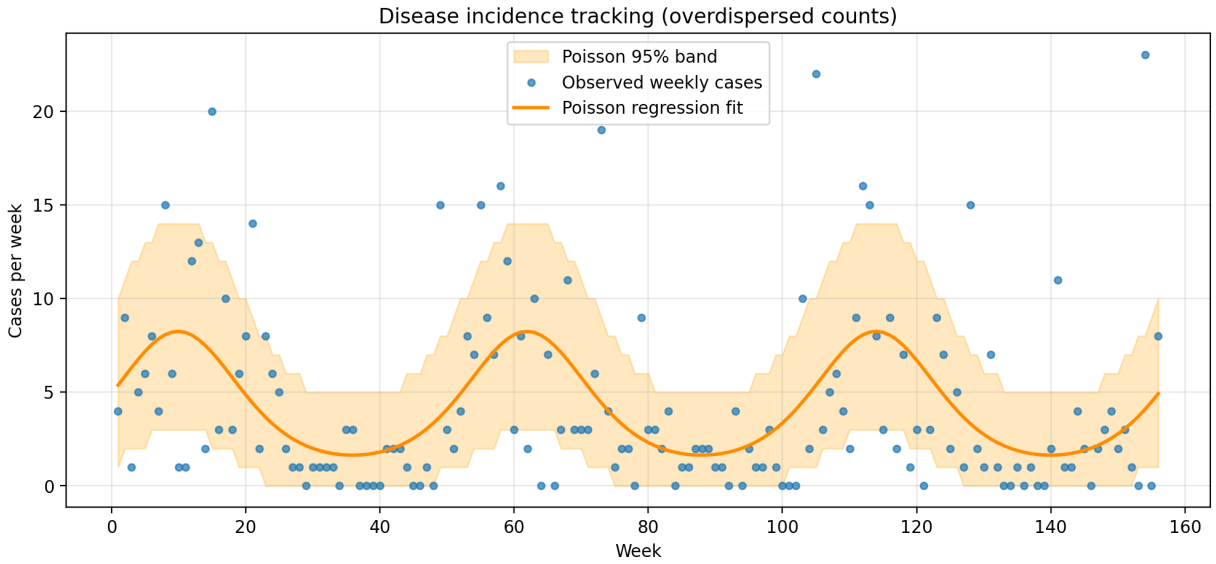


Figure 8: Weekly cases with the Poisson fit and its 95% band. Many weeks fall outside the band.

	β_0	$\beta_{\sin}$	$\beta_{\cos}$
True value	1.200	0.700	0.400
Estimate	1.302	0.753	0.292
Std. error, Poisson	0.045	0.061	0.057
Std. error, quasi-Poisson	0.093	0.125	0.118

Table 2: Poisson regression on overdispersed counts: the point estimates are reasonable but the Poisson standard errors are about half as large as they should be.

The dispersion is 4.24, and only 80% of the weeks fall inside the nominal 95% Poisson band. The quasi-Poisson standard errors are larger by $\sqrt{4.24} \approx 2.06$. A Negative Binomial model is the natural next step (Section 9.1).

7.6 E-commerce clickstream

Use case: relate engagement to session length. Here $\log \mu = \beta_0 + \beta_1 \cdot \text{minutes}$ and e^{β_1} is the multiplicative increase in expected clicks per extra minute. (In the first version `clicks` were generated independently of `session_time`, so the fitted "relationship" was an artefact.)

```
session_time = rng.exponential(scale=5, size=1000) # minutes
X = np.column_stack([np.ones(1000), session_time])
beta_true = np.array([0.5, 0.06])
clicks = rng.poisson(np.exp(X @ beta_true))

click_model = sm.GLM(clicks, X, family=sm.families.Poisson()).fit()
```

```
b0, b1 = click_model.params
print("true beta :", beta_true)
print("estimated beta:", np.round(click_model.params, 3), "| std errors:", np.round(click_model.
    bse, 3))
print("rate ratio per extra minute exp(beta1) =", round(np.exp(b1), 3))
print("dispersion :", round(dispersion(click_model), 3))

grid = np.linspace(0, session_time.max(), 200)
plt.figure(figsize=(10, 5))
plt.scatter(session_time, clicks + rng.uniform(-0.15, 0.15, clicks.size), s=8, alpha=0.3,
           label="Observed clicks (jittered)")
plt.plot(grid, np.exp(b0 + b1 * grid), color="purple", lw=2, label="Poisson regression fit")
plt.plot(grid, np.exp(beta_true[0] + beta_true[1] * grid), "g--", lw=2, label="True mean")
plt.title("E-commerce clickstream analysis")
plt.xlabel("Session time (minutes)")
plt.ylabel("Number of clicks")
plt.legend()
plt.show()
```

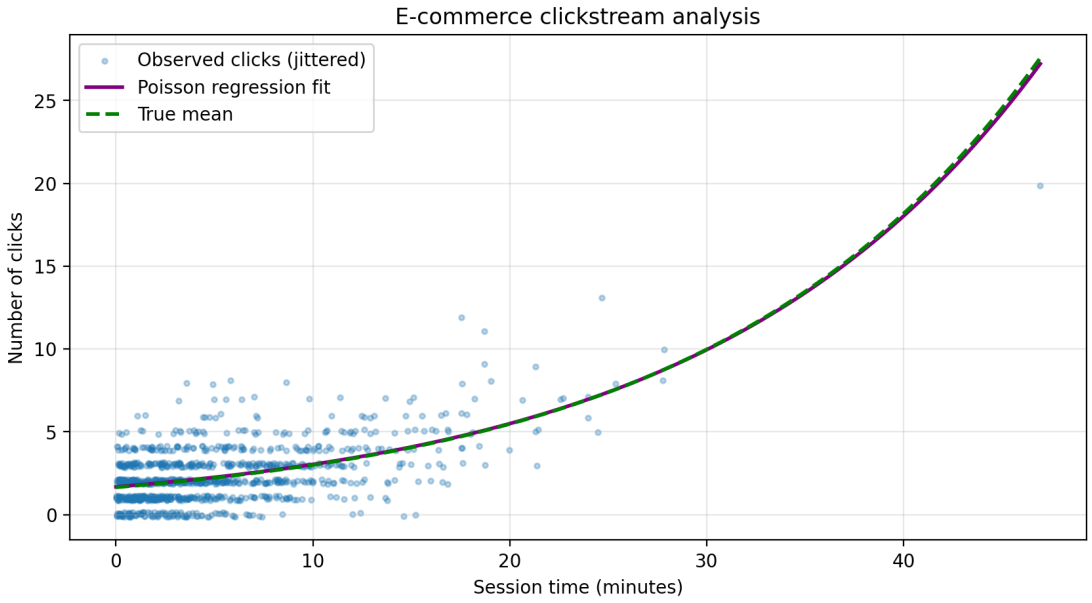


Figure 9: Clicks against session time (jittered), the fitted Poisson regression and the true mean.

The estimates $\hat{\beta} = (0.486, 0.063)$ with standard errors $(0.031, 0.004)$ are close to the truth $(0.5, 0.06)$. The rate ratio $e^{\hat{\beta}_1} = 1.065$: each additional minute multiplies the expected number of clicks by about 6.5%. The dispersion is 0.973.

8 Limitations

- Real data often violate independence, constant rate or equidispersion. Always check the dispersion before trusting Poisson standard errors.
- The synthetic data in this paper validate the *method* (parameter recovery, calibration), not any real-world claim about call centres, traffic, disease or clicks.
- The anomaly-detection example assumes the training period is free of attacks and that λ is constant; in practice λ may drift and should be re-estimated or modelled by regression.

9 Conclusion

The Poisson distribution is a compact but powerful model for count data. We gave complete proofs of its main properties, corrected the definition and interpretation of its hazard function, showed its links to the binomial, exponential and normal laws, and demonstrated Poisson regression and a Poisson tail-probability detector on five applications whose synthetic data contain a known signal. The accompanying notebook reproduces all results.

9.1 Future directions

1. **Overdispersion and excess zeros:** Negative Binomial and Zero-Inflated Poisson models, compared by AIC and residual diagnostics.
2. **Exposure offsets:** include log(exposure) via the `offset` argument when intervals or populations differ.
3. **Non-homogeneous Poisson process:** a time-varying rate $\lambda(t)$ and the corresponding waiting-time distribution.
4. **Real datasets:** replace synthetic data by public data (traffic counts, call-centre logs, surveillance data).
5. **Bayesian modelling:** Gamma–Poisson conjugate analysis and hierarchical models for uncertainty in λ.
6. **Deep learning:** Poisson negative log-likelihood as a loss for neural count models.
7. **Goodness of fit:** chi-square tests, rootograms and Q–Q plots.
8. **Interactive dashboard:** a slider for λ that updates the PMF/CDF and hazard live.

9.2 Educational implications

Understanding the Poisson distribution equips students and practitioners to model event counts, to recognise when its assumptions fail, and to move from the basic model to its regression and overdispersed extensions.

References

[1] W. Feller. *An Introduction to Probability Theory and Its Applications*, Vol. 2, 2nd ed. Wiley, 1971.

[2] N. L. Johnson, A. W. Kemp, S. Kotz. *Univariate Discrete Distributions*, 3rd ed. Wiley, 2005.

[3] J. F. Lawless. *Statistical Models and Methods for Lifetime Data*, 2nd ed. Wiley, 2003.

[4] P. McCullagh, J. A. Nelder. *Generalized Linear Models*, 2nd ed. Chapman and Hall, 1989.

[5] A. C. Cameron, P. K. Trivedi. *Regression Analysis of Count Data*. Cambridge University Press, 1998.

[6] S. Seabold, J. Perktold. statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*, 2010.

[7] A. Agresti. *Categorical Data Analysis*, 3rd ed. Wiley, 2013.

[8] A. Gelman, J. Hill. *Data Analysis Using Regression and Multilevel/Hierarchical Models.* Cambridge University Press, 2007.

[9] C. M. Bishop. *Pattern Recognition and Machine Learning.* Springer, 2006.